

ONLINE BOOKSTORE DATABASE

SQL / MySQL Database Project Report

Prepared by: Clement Jonathan

Technology: MySQL / SQL

Project Overview

This project presents the design and implementation of an Online Bookstore database using MySQL. The database manages books, customers, orders, payments and order activity logs. It demonstrates core SQL concepts including database creation, table relationships, data insertion, queries, joins, aggregation, triggers, stored procedures and user privileges.

1. Project Objectives

The main objectives of the project are to:

- Create a structured relational database for an online bookstore.
- Store and manage information about books, customers, orders and payments.
- Establish relationships between tables using primary and foreign keys.
- Use SQL queries to retrieve useful business information.
- Calculate total books sold and total revenue.
- Identify customers and products associated with orders and payments.
- Automate order activity tracking with database triggers.
- Use a stored procedure to retrieve a customer's orders.
- Demonstrate database user privileges for controlled access.

2. Database Structure

The database is named **bookstore** and contains five tables: books, customers, orders, payments and order_log.

Table	Purpose	Key relationship
books	Stores book title, author and price.	book_id – Primary Key
customers	Stores customer details.	customer_id – Primary Key
orders	Records customer book orders.	customer_id and book_id – Foreign Keys
payments	Stores payment information for orders.	order_id – Foreign Key
order_log	Records INSERT, UPDATE and DELETE activity on orders.	log_id – Primary Key

3. Database Creation and Table Design

The database is created with CREATE DATABASE and selected with USE. The tables are then created with appropriate data types and constraints.

Key SQL structure:

```
CREATE DATABASE bookstore;  
USE bookstore;
```

```
CREATE TABLE books (  
  book_id INT PRIMARY KEY,  
  title VARCHAR(255),  
  author VARCHAR(255),  
  price DECIMAL(10,2)  
);
```

The orders and payments tables use foreign keys to maintain relationships with customers, books and orders. This helps maintain referential integrity within the database.

4. Data Insertion

Sample data was inserted into the books, customers, orders and payments tables. Five books, five customers, five initial orders and five payments were provided. An additional order was later inserted to test the order trigger.

Example:

```
INSERT INTO orders VALUES (6, 1, 2, 1, '2023-06-06');
```

5. SQL Queries and Results

The project demonstrates several types of SQL queries.

Retrieve all customers

```
SELECT * FROM customers;
```

Returns all customer records.

List books and authors

```
SELECT title, author FROM books;
```

Displays book titles together with their authors.

Calculate books sold

```
SELECT SUM(quantity) AS total_books_sold FROM orders;
```

Based on the initial five orders, 9 books were sold.

Find customers who ordered

```
SELECT DISTINCT c.name FROM customers c JOIN orders o ON c.customer_id = o.customer_id;
```

Returns customers with recorded orders.

Calculate total revenue

```
SELECT SUM(amount) AS total_revenue FROM payments;
```

The five initial payments total 111.91.

Find highest payment

```
SELECT c.name, p.amount JOIN orders o ON c.customer_id = o.customer_id JOIN payments p ON o.order_id = p.order_id ORDER BY p.amount DESC LIMIT 1;
```

Customer 4 has the highest recorded payment of 31.98.

Find most expensive book

```
SELECT title, price FROM books ORDER BY price DESC LIMIT 1;
```

Book 4 is the most expensive at 15.99.

6. Triggers

Three triggers were created on the orders table to automatically record INSERT, UPDATE and DELETE operations in the order_log table.

Trigger	Event	Action
after_order_insert	INSERT	Logs INSERT
after_order_update	UPDATE	Logs UPDATE
after_order_delete	DELETE	Logs DELETE

After inserting order 6, `SELECT * FROM order_log` can be used to verify that the INSERT action was automatically recorded.

7. Stored Procedure

A stored procedure named `GetCustomerOrders` was created to retrieve orders belonging to a specified customer. It joins the orders and books tables and accepts a customer ID as an input parameter.

```
CALL GetCustomerOrders(1);
```

This provides a reusable way to retrieve a customer's order ID, book title, quantity and order date.

8. Database User and Privileges

A separate database user named `book_user` was created and granted `SELECT`, `INSERT` and `ALTER` privileges on the bookstore database. This demonstrates basic access control and the principle of giving users only the permissions needed for their tasks.

```
CREATE USER 'book_user'@'localhost' IDENTIFIED BY '12345';  
GRANT SELECT, INSERT, ALTER ON bookstore.* TO 'book_user'@'localhost';  
FLUSH PRIVILEGES;
```

For a real production database, a stronger password and more carefully scoped privileges should be used.

9. Theory Questions and Answers

9.1 Difference Between SQL and MySQL

SQL (Structured Query Language) is a language used to create, retrieve, update and manage data in databases. **MySQL** is a relational database management system (RDBMS) that uses SQL. In simple terms, SQL is the language while MySQL is the software/database system.

9.2 Why Are Triggers Created in a Database?

Triggers automatically execute predefined instructions when events such as `INSERT`, `UPDATE` or `DELETE` occur. They can automate repetitive tasks, support data integrity, track changes and improve database auditing.

9.3 Difference Between Primary Key and Foreign Key

A **primary key** uniquely identifies a record and cannot contain duplicate or `NULL` values. A **foreign key** creates a relationship by referencing a primary key in another table and may contain duplicate values. For example, `customer_id` is a primary key in `customers` and a foreign key in `orders`.

9.4 Difference Between DBMS and RDBMS

A **DBMS** is software used to store and manage data. An **RDBMS** is a DBMS that organizes data into related tables and commonly uses keys to establish relationships. Examples include MySQL, Oracle, PostgreSQL and SQL Server.

10. Key Findings

- The database successfully models the main operations of an online bookstore.
- The initial five orders represent 9 books sold.
- The initial five payments generate total revenue of 111.91.
- The highest initial payment is 31.98 from Customer 4.
- Book 4 has the highest price at 15.99.
- Triggers provide automatic logging of order activities.
- The stored procedure provides reusable customer-order retrieval.

- Primary and foreign keys establish relationships and support data integrity.

11. Conclusion

The Online Bookstore Database was developed using MySQL and demonstrates fundamental and intermediate SQL database concepts. The project includes Books, Customers, Orders, Payments and Order Log tables linked through primary and foreign keys. SQL queries were used to retrieve and summarize information, while triggers, a stored procedure and user privileges were implemented to improve automation, reusability, auditing and access control. Overall, the project demonstrates how a relational database can support the management of an online bookstore's core operations.